

LAPORAN PRAKTIKUM STRUKTUR DATA



Oleh:
FAWWAZ KHALID
2511532004

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T
ASISTEN LABOR : M GALID

FAKULTAS TEKNOLOGI INFORMASI

PEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 2026

KATA PENGANTAR

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah Pemrograman Dasar dengan Java. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2025

Tim Penyusun

FAWWAZ KHALID

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Latar Belakang Praktikum.....	1
1.3 Rumusan Masalah.....	1
1.4 Tujuan Praktikum.....	1
BAB II PENULISAN LAPORAN PRAKTIKUM.....	2
2.1 Alat dan Bahan.....	2
2.2 Landasan Teori.....	2
2.3 Langkah Kerja.....	2
2.4 Hasil dan Analisis.....	3
BAB III KESIMPULAN.....	4
3.1 Ringkasan.....	4
DAFTAR PUSTAKA.....	4

BAB I

PENDAHULUAN

1.1 Pengertian ADT

Tipe Data Abstrak (Tda) Atau Lebih Dikenal Dalam [Bahasa Inggris](#) Sebagai **Abstract Data Type (Adt)** Merupakan Model [Matematika](#) Yang Merujuk Pada Sejumlah Bentuk Struktur Data Yang Memiliki Kegunaan Atau Perilaku Yang Serupa; Atau Suatu [Tipe Data](#) Dari Suatu [Bahasa Pemrograman](#) Yang Memiliki Semantik Yang Serupa. Tipe Data Abstrak Umumnya Didefinisikan Tidak Secara Langsung, Melainkan Hanya Melalui Operasi Matematis Tertentu Sehingga Membutuhkan Penggunaan Tipe Data Tersebut Meski Dengan Risiko Kompleksitas Yang Lebih Tinggi Atas Operasi Tersebut.

BAB II

PENULISAN LAPORAN PRAKTIKUM

2.1 Alat dan Bahan

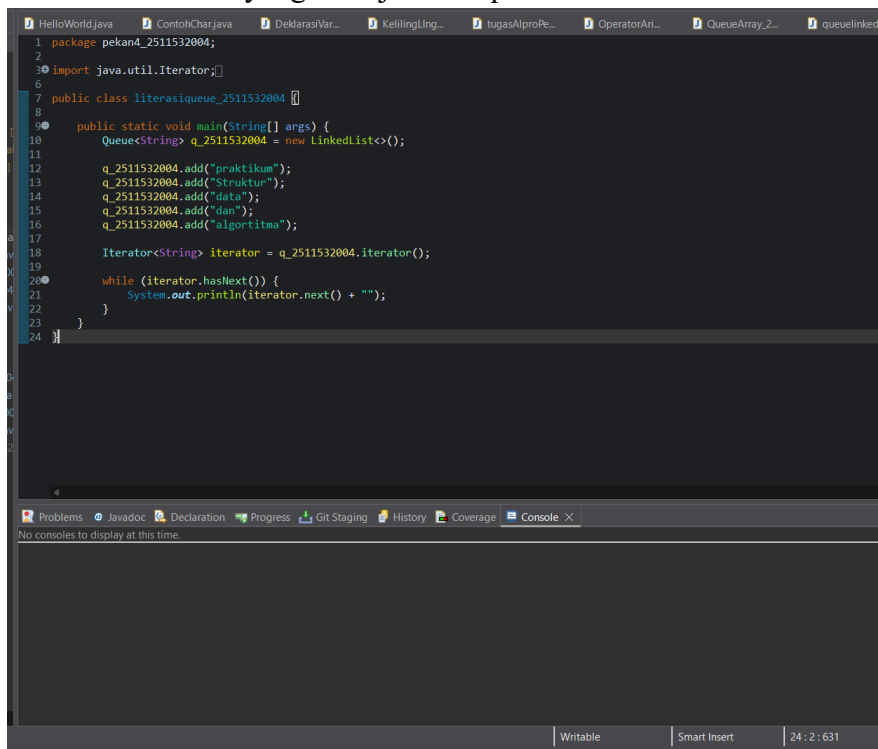
1. Perangkat baik komputer maupun laptop
2. Teks editor atau *IDE* (misalnya NetBeans, Eclipse, atau IntelliJ IDEA)

2.2 Landasan Teori

1. Tipe data sebuah variabel adalah kumpulan nilai yang dapat dimuat oleh variabel ini. Misalnya sebuah tipe boolean hanya bernilai TRUE atau FALSE, tidak boleh nilai yang lain. TDA adalah suatu model matematika, disertai sekumpulan operasi terhadap model itu

2.3 Langkah Kerja

1. Membuat proyek JAVA
2. Membuat file baru di dalam Proyek JAVA tersebut
3. Masukkan kode yang ingin dijalankan
contoh kode yang dikerjakan di pekan 4:



```
1 package pekan4_2511532004;
2
3 import java.util.Iterator;
4
5
6
7 public class literasiqueue_2511532004 {
8
9     public static void main(String[] args) {
10         Queue<String> q_2511532004 = new LinkedList<>();
11
12         q_2511532004.add("praktikum");
13         q_2511532004.add("Struktur");
14         q_2511532004.add("data");
15         q_2511532004.add("dan");
16         q_2511532004.add("algoritma");
17
18         Iterator<String> iterator = q_2511532004.iterator();
19
20         while (iterator.hasNext()) {
21             System.out.println(iterator.next() + "");
22         }
23     }
24 }
```

The screenshot shows an IDE window with a Java file named `literasiqueue_2511532004.java`. The code defines a package, imports `java.util.Iterator`, and creates a class `literasiqueue_2511532004`. Inside the `main` method, a `LinkedList` queue is created and populated with the words "praktikum", "Struktur", "data", "dan", and "algoritma". An `Iterator` is then used to iterate through the queue and print each element followed by a space.

```

1 package pekan4_2511532004;
2
3 public class queuearraydriver_2511532004 {
4     public static void main(String[] args) {
5         QueueArray_2511532004 queue = new QueueArray_2511532004(1000);
6         queue.enqueue_2004(10);
7         queue.enqueue_2004(20);
8         queue.enqueue_2004(30);
9         queue.enqueue_2004(40);
10        System.out.println("item depan" + queue.front_2004());
11        System.out.println("item belakang" + queue.rear_2004);
12        System.out.println("tampilkan queue");
13        queue.display_2004();
14        System.out.println();
15        System.out.println(queue.dequeue_2511532004() + "dihapus dari queue");
16        System.out.println("item depan" + queue.front_2004());
17        System.out.println("item belakang" + queue.rear_2004);
18        System.out.println("tampilkan queue setelah satu data di hapus");
19        queue.display_2004();
20
21    }
22 }
23
24 }
25

```

Problems Javadoc Declaration Progress Git Staging History Coverage Console X

No consoles to display at this time.

```

1 package pekan4_2511532004;
2
3 public class QueueArray_2511532004 {
4
5     int front_2004, rear_2004, size_2004;
6     int capacity_2004;
7     int array_2004[];
8
9     public QueueArray_2511532004(int capacity_2004) {
10        this.capacity_2004 = capacity_2004;
11        front_2004 = this.size_2004 = 0;
12        rear_2004 = capacity_2004 - 1;
13        array_2004 = new int[this.capacity_2004];
14    }
15
16    boolean isFull_2004(QueueArray_2511532004 queue_2004) {
17        return (queue_2004.size_2004 == queue_2004.capacity_2004);
18    }
19
20    boolean isEmpty_2004(QueueArray_2511532004 queue_2004) {
21        return (queue_2004.size_2004 == 0);
22    }
23
24    void enqueue_2004(int item_2004) {
25        if (isFull_2004(this))
26            return;
27
28        this.rear_2004 = (this.rear_2004 + 1) % this.capacity_2004;
29        this.array_2004[this.rear_2004] = item_2004;
30        this.size_2004 = this.size_2004 + 1;
31
32        System.out.println(item_2004 + " enqueued to queue");
33    }
34
35 }
36

```

Problems Javadoc Declaration Progress Git Staging History Coverage Console X

No consoles to display at this time.

```

1 package pekan4_2511532004;
2 import java.util.Queue;
3 import java.util.LinkedList;
4
5 public class queueLinkedList_2511532004 {
6     public static void main(String[] args) {
7         Queue<Integer> q_2511532004 = new LinkedList<>();
8         for (int i = 0; i < 6; i++)
9             q_2511532004.add(i);
10
11         System.out.println("elemen antrian " + q_2511532004);
12
13         int hapus = q_2511532004.remove();
14         System.out.println("hapus elemen " + hapus);
15         System.err.println(q_2511532004);
16
17         int depan_2511532004 = q_2511532004.peek();
18         System.out.println("kepala antrian = " + depan_2511532004);
19
20         int banyak_2511532004 = q_2511532004.size();
21         System.out.println("Size antrian = " + banyak_2511532004);
22     }
23 }
24

```

Problems Javadoc Declaration Progress Git Staging History Coverage Console X

No consoles to display at this time.

```

1 package pekan4_2511532004;
2 import java.util.LinkedList;
3
4
5
6
7 public class reversedata_2511532004 {
8     public static void main(String[] args) {
9         Queue<Integer> q_2511532004 = new LinkedList<Integer>();
10         q_2511532004.add(1);
11         q_2511532004.add(2);
12         q_2511532004.add(3);
13
14         System.out.println("sebelum reverse" + q_2511532004);
15         Stack<Integer> s = new Stack<Integer>();
16         while (!q_2511532004.isEmpty()) {
17             s.push(q_2511532004.remove());
18         }
19         while (!s.isEmpty()) {
20             q_2511532004.add(s.pop());
21         }
22         System.out.println("sesudah reverse " + q_2511532004);
23     }
24

```

Problems Javadoc Declaration Progress Git Staging History Coverage Console X

No consoles to display at this time.

2.4 Hasil dan Analisis

Jika kode berjalan seperti yang kita inginkan maka:

1. Menyimpan file
2. Mencatat hasil output

Jika kode gagal maka:

1. Mencari kode yang salah lalu coba jalankan ulang

BAB III

KESIMPULAN

3.1 Ringkasan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Deklarasi variabel pada Java harus disesuaikan dengan tipe data
2. Bisa menggunakan queue, stack, linkedlist dan arraylist untuk fungsi berbeda

DAFTAR PUSTAKA

1. Tipe data abstrak-wikipedia
https://id.wikipedia.org/wiki/Tipe_data_abstrak

